Git Core Commands & GitHub

1. Core Git Commands

Command	Purpose	Common Syntax
`git init`	Initializes a new, empty Git repository in the current folder by creating a hidden `.git` directory	`git init`
`git clone`	Creates a full local copy of an existing remote repository, including its entire history	`git clone <repo-url>`
`git commit`	Permanently records the currently staged changes as a new snapshot in the local repository	`git commit -m "message"`
`git push`	Uploads local commits to a remote repository so others can access them	`git push origin <branch-name>`
`git pull`	Downloads changes from a remote repository and merges them into the current local branch	`git pull origin <branch-name>`
`git branch`	Lists, creates, or deletes branches (pointers to specific commits)	`git branch <branch-name>`
`git merge`	Integrates the commit history of one branch into another	`git merge <branch-name>`

Key point to remember: `git init` / `git clone` establish a repository; `git commit` records history locally; `git push` / `git pull` synchronize that history with a remote server; `git branch` / `git merge` manage parallel lines of development within that history.

2. GitHub Fundamentals

2.1 Introduction to GitHub / What is GitHub?

- **GitHub** is a **web-based hosting platform** built specifically for repositories that use Git as their version control system.
- Founded in 2008, GitHub took Git's command-line-based, local-first system and wrapped it with

a **graphical, cloud-based interface**, along with a suite of collaboration tools.

- It is currently owned by **Microsoft** (acquired in 2018) and is the most widely used Git-hosting platform in the world, home to hundreds of millions of repositories, including massive open-source projects (Linux kernel mirror, VS Code, TensorFlow, React, etc.).
- Beyond just storing code, GitHub functions as a **social and collaborative hub for developers** — it allows people to discover projects, discuss issues, propose changes, review each other's code, and contribute to software they don't own, all through a structured, permission-based workflow.
- Other comparable platforms exist (GitLab, Bitbucket, SourceForge), but GitHub remains the dominant choice, especially for open-source development.

2.2 Git vs GitHub

Basis	Git	GitHub
What it is	A version control **software/tool**	A cloud-based **hosting service/platform** for Git repositories
Installation	Installed locally on your computer	Accessed via a web browser (or its own CLI/desktop tools); no installation required to use
Internet requirement	Not required — works fully offline for local operations	Required to access, push, pull, or collaborate
Created by	Linus Torvalds (2005)	Chris Wanstrath, PJ Hyett, Tom Preston-Werner, Scott Chacon (2008)
Ownership	Open-source, community-maintained	Owned by Microsoft
Core function	Tracks changes to files/history	Hosts repositories and adds collaboration features on top of Git
Extra features	None beyond version control itself	Pull requests, issues, project boards, GitHub Actions (CI/CD), wikis, discussions, code review tools
Analogy	Git is like the engine of a car	GitHub is like the car showroom/dealership where people can browse, share, and collaborate around that engine

Important distinction to remember for exams/interviews: You can use Git *without* ever using GitHub (purely locally). GitHub, however, cannot exist without Git — it is built entirely on top of it.

2.3 Benefits of GitHub

- **Centralized collaboration hub**: Provides a single, accessible online location where an entire team (or the entire world, for open-source projects) can view, contribute to, and discuss a codebase.
- **Remote backup**: Since code is hosted on GitHub's servers, it serves as an additional off-site backup beyond just local machines.
- **Pull requests and code review**: Enables structured proposal and review of changes before they're merged into the main codebase, improving code quality and catching issues early.
- **Issue tracking**: Built-in system for reporting bugs, requesting features, and tracking tasks, all linked directly to the relevant code and commits.
- **Access control**: Fine-grained permission settings determine who can view, edit, or administer a repository.
- **Portfolio and visibility**: Developers use public GitHub profiles to showcase their work, which has become a standard part of job applications in the software industry.
- **Integration with CI/CD**: Through **GitHub Actions**, code can be automatically tested, built, and deployed whenever changes are pushed, streamlining the development pipeline.
- **Community and open-source ecosystem**: Millions of open-source projects rely on GitHub, allowing developers worldwide to contribute to software they don't own via forks and pull requests.
- **Documentation hosting**: Features like README files and Wikis make it easy to document a project directly alongside its code.

2.4 GitHub Features

- **Repositories**: The core unit of storage — holds all project files, folders, and version history.
- **Issues**: A tracking system for bugs, feature requests, and general tasks, which can be assigned, labeled, and linked to specific commits or pull requests.
- **Pull Requests (PRs)**: A formal mechanism to propose that changes from one branch (or fork) be merged into another, with built-in discussion and review tools.
- **Actions (CI/CD)**: Automation workflows that can run tests, build code, or deploy applications automatically in response to events (e.g., a push or pull request).
- **Projects/Project Boards**: Kanban-style boards for organizing and tracking work items.
- **Wikis**: Space for writing extended project documentation directly within the repository's GitHub page.
- **Discussions**: A forum-like space for broader conversations that don't necessarily map to a specific issue or bug.
- **Forking**: Allows any user to create their own copy of someone else's repository under their own account, to freely experiment or propose changes.
- **GitHub Pages**: Allows hosting static websites directly from a repository, often used for documentation sites or portfolios.
- **Security features**: Includes dependency vulnerability scanning (Dependabot), secret scanning, and code scanning tools.

- **Releases**: A way to package and distribute specific, tagged versions of software (e.g., v1.0, v2.0) along with release notes and downloadable files.

3. GitHub Repository Management

3.1 Creating a GitHub Account

- To use GitHub, a user must first sign up at github.com with a username, email address, and password.
- Account creation is free for individual use (with paid tiers available offering additional features for teams and organizations, like advanced security tools or more compute minutes for Actions).
- After signing up, users typically set up a profile (with a photo, bio, and pinned repositories) that acts as their public developer identity, visible to anyone browsing GitHub.
- Two-factor authentication (2FA) is strongly recommended (and increasingly required by GitHub) to secure the account, since it controls access to potentially critical codebases.

3.2 Creating a GitHub Repository

- Repositories on GitHub can be created directly through the web interface by clicking "New Repository," where you specify:
  - **Repository name**
  - **Description** (optional but recommended)
  - **Visibility**: Public or Private
  - Optional initialization files: a README, a `.gitignore` template, and/or a license file
- Once created, GitHub provides a unique URL for the repository (e.g., `github.com/username/repository-name`), which is used to clone it or connect a local repository to it.
- A repository can also be created first *locally* (via `git init`) and connected to a newly created, empty GitHub repository afterward — this is a very common real-world workflow.

3.3 Public Repository vs Private Repository

Basis	Public Repository	Private Repository
Visibility	Visible to anyone on the internet, whether they have a GitHub account or not	Visible only to the owner and explicitly invited collaborators
Use case	Open-source projects, portfolios, learning projects	Proprietary/commercial code, sensitive projects, work-in-progress
Contribution	Anyone can fork it and propose changes via pull requests (though merging still requires owner/maintainer approval)	Only invited collaborators can view or contribute
Cost	Always free	Free with reasonable limits on personal accounts; larger teams/organizations may require paid plans for advanced features
Search engines	Can be discovered via GitHub search and often indexed by search engines	Not discoverable by non-collaborators

3.4 Repository Structure

A typical GitHub repository contains:

- **Source code files and folders**, organized according to the project's structure/architecture.
- `.git` **folder** (hidden): Contains all version history and Git metadata — this exists whether the repo is local or on GitHub, though GitHub's web interface doesn't expose it directly.
- `README.md`: The landing page/documentation shown by default when someone visits the repository on GitHub.
- `.gitignore`: Specifies files/folders that Git should intentionally *not* track (e.g., build artifacts, secret keys, dependency folders).
- `LICENSE`: A file specifying the legal terms under which others may use, modify, or distribute the code.
- **Branches**: A repository typically has a default branch (commonly `main`) along with any number of additional feature, development, or release branches.
- **Commit history**: Every change ever made, viewable through GitHub's "Commits" tab.
- **Issues/Pull Requests tabs**: Not files per se, but core structural components of how the repository is managed and collaborated on.

---

4. Connecting Git with GitHub

4.1 Connecting Local Repository to GitHub Repository

- If you already have a local Git repository (created via `git init`) and want to connect it to a newly created, empty repository on GitHub, you use:

```
git remote add origin <repository-url>
```

- This command doesn't transfer any data by itself — it simply tells your local repository, "there exists a remote named `origin`, located at this URL, that you can push to or pull from."
- After adding the remote, you push your existing local commits up to GitHub for the first time using:

```
git push -u origin main
```

The `-u` flag sets up "upstream tracking," linking your local `main` branch to the remote's `main` branch, so future `git push`/`git pull` commands don't require specifying the remote and branch name every time.

4.2 Understanding Remote Repository URLs

- A remote URL tells Git exactly where a remote repository is located and how to communicate with it. GitHub provides two main formats:
  - **HTTPS URL**: e.g., `https://github.com/username/repository.git` — simpler to set up, but typically requires entering credentials (username/password or a personal access token) on each push, unless credentials are cached.
  - **SSH URL**: e.g., `git@github.com:username/repository.git` — requires an SSH key pair to be set up and added to your GitHub account beforehand, but afterward allows for authentication without repeatedly entering credentials.
- The choice between HTTPS and SSH is largely about convenience and security preference — both achieve the same fundamental goal of linking your local repository to its remote counterpart.
- You can view your current remote configuration using `git remote -v`, which lists the name (e.g., `origin`) alongside its associated fetch and push URLs.

4.3 Synchronizing Local and Remote Repositories

- Keeping local and remote repositories in sync is an ongoing process throughout development, primarily handled through three commands:
  - `git push`: Sends your local commits up to the remote, making them visible/accessible to collaborators.
  - `git pull`: Brings new commits from the remote down into your local repository, merging them into your current branch.
  - `git fetch`: Downloads new commits and history from the remote without automatically merging them, letting you review incoming changes before integrating them.
- A common real-world issue arises when both the local and remote repositories have diverged — meaning both have new commits the other doesn't have. In this case, a plain `git push` will be rejected by GitHub (to prevent accidentally overwriting a collaborator's work), and you must first `git pull` (resolving any conflicts if they arise) before pushing again.
- Best practice is to **pull frequently**, especially before starting new work, to minimize the chances of large, complicated divergences and conflicts.

---

5. GitHub Collaboration

5.1 Forking a Repository

- A **fork** is a personal copy of someone else's repository, created under your own GitHub account.
- Forking allows you to freely experiment with, modify, or build upon a project without needing direct write access to the original repository.
- Once forked, you clone *your own fork* to your local machine, make changes, commit them, and push them back to your fork on GitHub.
- Forking is the standard mechanism for contributing to open-source projects where you are not an official collaborator — since you can't push directly to the original repository, you propose your changes back via a pull request instead.

5.2 Pull Requests (PR)

- A **Pull Request** is a formal request asking the maintainers of a repository to review and merge changes from one branch (often a branch on your fork) into another branch (typically the original project's main branch).
- A PR isn't just a merge mechanism — it's a **discussion and review space**: maintainers and other contributors can leave comments on specific lines of code, request changes, approve the PR, or reject it entirely.
- PRs display a full diff of proposed changes, along with automated checks (if configured via GitHub

Actions), such as whether tests pass.
- PRs can be updated after creation — if a reviewer requests changes, the contributor can simply push new commits to the same branch, and the PR automatically updates to reflect them.

5.3 Code Review Process

- Code review is the practice of having other developers examine proposed changes before they are merged into the main codebase.
- On GitHub, this happens directly within a pull request: reviewers can leave inline comments on specific lines, request changes, approve, or explicitly reject a PR.
- The process typically improves code quality, catches bugs early, spreads knowledge of the codebase among team members, and enforces coding standards/conventions.
- Many teams configure **branch protection rules** requiring at least one (or more) approving reviews before a PR is allowed to be merged, preventing unreviewed code from directly entering the main branch.

5.4 Merging Pull Requests

- Once a pull request has been reviewed and approved, it can be merged into the target branch, typically via a button directly in the GitHub interface.
- GitHub offers a few different merge strategies:
  - **Merge commit**: Preserves the full commit history of the feature branch, combining it with the target branch via a new merge commit (similar to a standard `git merge`).
  - **Squash and merge**: Combines all the commits from the feature branch into a single, clean commit before merging — useful for keeping the main branch's history tidy.
  - **Rebase and merge**: Reapplies the feature branch's commits individually on top of the target branch, resulting in a linear history without an explicit merge commit.
- After merging, it's common practice to delete the now-redundant feature branch, since its purpose has been fulfilled.

5.5 Collaboration Workflow

A typical end-to-end collaboration workflow on GitHub looks like this:

1. **Fork** (if you're an external contributor) or directly **clone** the repository (if you're an official collaborator).
2. **Create a new branch** for the specific feature or fix you're working on.
3. Make changes locally, then `git add` and `git commit` them.

4. **Push** the branch to your fork (or the shared repository, if you have write access).
5. Open a **Pull Request** on GitHub, describing what the change does and why.
6. Collaborators/maintainers **review** the code, leaving comments or requesting changes as needed.
7. The contributor addresses feedback by pushing additional commits to the same branch.
8. Once approved, the PR is **merged** into the main branch.
9. The feature branch is deleted, and collaborators **pull** the updated main branch to stay in sync.

6. Repository Documentation and Configuration

6.1 README.md

- The **README** file is the first thing most people see when visiting a repository — GitHub automatically renders and displays it directly on the repository's main page.
- Written in **Markdown** (`.md` extension), allowing for formatted headings, lists, code blocks, images, tables, and links.
- A good README typically includes:
  - Project name and a brief description of what it does.
  - Installation/setup instructions.
  - Usage examples.
  - Contribution guidelines.
  - License information.
- Serves both as **documentation for users** (how to use the project) and **onboarding material for new contributors** (how to get started developing).

6.2 .gitignore

- The `.gitignore` file tells Git which files or folders it should **intentionally not track**, even if they exist in the working directory.
- Common candidates for `.gitignore` include:
  - Dependency folders (e.g., `node_modules/`)
  - Build/compiled output (e.g., `dist/`, `build/`)
  - Environment/configuration files containing secrets (e.g., `.env`)
  - OS-specific or editor-specific files (e.g., `.DS_Store`, `.vscode/`)
  - Log files and temporary files
- Each line in a `.gitignore` file specifies a pattern; files matching that pattern are excluded from `git add` / `git status` / `git commit` entirely.
- Important nuance: `.gitignore` only prevents *untracked* files from being tracked — if a file was

already being tracked before being added to `.gitignore`, it must be explicitly removed from tracking (e.g., using `git rm --cached <file>`).
- GitHub provides ready-made `.gitignore` templates for many popular languages/frameworks (Node.js, Python, Java, etc.) that can be selected during repository creation.

6.3 License Files

- A **LICENSE** file specifies the legal terms under which others are permitted to use, modify, and distribute your code.
- Without an explicit license, default copyright law generally means others technically do not have legal permission to reuse your code, even if the repository is public.
- Common open-source license types include:
  - **MIT License**: Very permissive — allows almost any use, including commercial, with minimal restrictions (just requires attribution).
  - **Apache License 2.0**: Permissive, similar to MIT, but includes explicit patent grant clauses.
  - **GPL (GNU General Public License)**: A "copyleft" license requiring that derivative works also be released under the same license.
- GitHub allows selecting a license template directly during repository creation, or adding one later via the web interface.

7. GitHub Best Practices

7.1 Meaningful Commit Messages

- Commit messages form a permanent, readable record of a project's evolution — vague messages like "fix stuff" or "update" provide little value to future developers (including yourself).
- Good practice generally follows this structure:
  - A short summary line (ideally under ~50 characters), written in the imperative mood (e.g., "Add login validation" rather than "Added" or "Adding").
  - Optionally, a blank line followed by a more detailed explanation of *why* the change was made, if the summary alone isn't sufficient.
- Meaningful commit messages make it significantly easier to use tools like `git log`, `git blame`, and `git bisect` to trace the history and cause of bugs.

7.2 Branch Naming Conventions

- Consistent branch naming helps teams quickly understand the purpose of a branch at a glance. Common conventions include prefixes indicating the type of work:
  - `feature/branch-name` — for new features
  - `bugfix/branch-name` or `fix/branch-name` — for bug fixes
  - `hotfix/branch-name` — for urgent fixes applied directly to production
  - `release/branch-name` — for preparing a new release
- Descriptive, hyphen-separated names (e.g., `feature/user-authentication`) are preferred over vague names (e.g., `new-branch` or `test123`).

7.3 Repository Organization

- Maintaining a clean, logical folder structure makes a project significantly easier to navigate and maintain.
- Best practices generally include:
  - Keeping a clear separation between source code, documentation, tests, and configuration files.
  - Including a well-written README at the root of the repository.
  - Using `.gitignore` properly to avoid cluttering the repository with unnecessary or sensitive files.
  - Applying consistent naming conventions for files and folders.
  - Regularly cleaning up merged or stale branches to avoid clutter.
  - Documenting contribution guidelines (often in a `CONTRIBUTING.md` file) for projects that accept outside contributions.

8. GitHub Practicals

8.1 Create GitHub Account

- Visit github.com → Sign up → Provide username, email, password → Verify email → (Recommended) Enable two-factor authentication for security.

8.2 Create GitHub Repository

- Click "New Repository" on GitHub → Enter repository name and description → Choose Public or Private → Optionally initialize with a README, `.gitignore`, and license → Click "Create repository."

8.3 Connect Local Repository to GitHub (`git remote add origin`)

```
git remote add origin https://github.com/username/repository.git
```

- This links your existing local repository to the newly created (empty) GitHub repository, under the alias `origin`.

8.4 Push Code to GitHub (`git push`)

```
git push -u origin main
```

- Uploads your local commits to GitHub for the first time, and sets up tracking between your local `main` branch and the remote `main` branch. Subsequent pushes can simply use `git push`.

8.5 Pull Code from GitHub (`git pull`)

```
git pull origin main
```

- Downloads any new commits from the GitHub repository and merges them into your current local branch, keeping your local copy up to date with collaborators' changes.

8.6 Fetch Remote Changes (`git fetch`)

```
git fetch origin
```

- Downloads new commits/history from GitHub without merging them into your working files, allowing you to review incoming changes (e.g., via `git log origin/main`) before deciding to integrate them.

8.7 Create Pull Request

- Push your feature branch to GitHub → Navigate to the repository on GitHub → Click "Compare & pull request" (or manually go to the "Pull Requests" tab → "New Pull Request") → Select the base branch (e.g., `main`) and the compare branch (your feature branch) → Add a title and description explaining the change → Click "Create Pull Request."

8.8 Merge Pull Request

- After the PR has been reviewed and approved (and any required checks have passed) → Click "Merge Pull Request" on GitHub → Choose a merge strategy (merge commit, squash, or rebase) → Confirm the merge → Optionally delete the now-merged feature branch.

8.9 Create README.md

- Create a file named `README.md` in the root of the repository (either directly on GitHub via "Add file" → "Create new file," or locally, then committed and pushed).
- Write project documentation using Markdown syntax; GitHub automatically renders it on the repository's main page.

8.10 Create .gitignore

- Create a file named `.gitignore` in the root of the repository.
- List file/folder patterns to be excluded from version control, one per line (e.g., `node_modules/`, `.env`, `*.log`).
- GitHub also offers pre-built `.gitignore` templates for specific languages/frameworks during repository creation.

8.11 Publish Project on GitHub

- Combining the steps above: initialize or clone a local repository → make and commit changes → create a corresponding repository on GitHub → connect the local repository via `git remote add origin` → push the code using `git push -u origin main` → optionally add a README, `.gitignore`, and license → the project is now live and accessible on GitHub, ready for others to view, clone, fork, or contribute to.